
Table of Contents

.....	1
inertias and frames	2
control	3
initial kinematics from part 2	3
Simulate	4
Plot 5 orbit case	4
wheel speed	5
Euler angles: body to ECI	5
Quaternion: body to ECI	6
Euler angles: body to LVLH	6
Quaternion: body to LVLH	6
Angular velocity wrt ECI, body components	6
Angular velocity wrt LVLH, body components	6
Commanded moment	7
Plot 300 second case	7
wheel speed	8
Euler angles: body to ECI	8
Quaternion: body to ECI	8
Euler angles: body to LVLH	8
Quaternion: body to LVLH	8
Angular velocity wrt ECI, body components	9
Angular velocity wrt LVLH, body components	9
Commanded moment	9

```
clear; close all; clc;
```

```
% run part 1 script to get mass props
```

```
Part1_MassProps
```

```
Detumble total mass = 640 kg
```

```
Detumble COM =
```

```
0      0      0
```

```
Detumble inertia matrix =
```

```
426.6667      0      0
      0 426.6667      0
      0      0 426.6667
```

```
Normal total mass = 640 kg
```

```
Normal COM =
```

```
0      0      0.2344
```

```
Normal inertia matrix =
```

```
812.0396      0      0
      0 545.3729      0
      0      0 627.7083
```

inertias and frames

one reaction wheel pointing in each body axis dir. same setup as "Example - Simple Three Axis Setup" from class

```
num_wheels = 3;

% rotation matrices for each reaction wheel
C_b_w1 = [1 0 0;
          0 1 0;
          0 0 1];

C_b_w2 = [0 0 1;
          1 0 0;
          0 1 0];

C_b_w3 = [0 1 0;
          0 0 1;
          1 0 0];

% RW rotational inertias (all RWs identical)
mw = 1; % mass of rw, kgs
mw1 = mw;
mw2 = mw;
mw3 = mw;

Is = 1.2; % kg/m^2
It = 0.6; % kg/m^2

I_w1 = [Is 0 0;
        0 It 0;
        0 0 It];
I_w2 = I_w1;
I_w3 = I_w1;

% s/c inertia
I_sc = n.sat.I;

% wheel pointing vectors in body
r1 = [1;0;0];
r2 = [0;1;0];
r3 = [0;0;1];

% form vectors of RW props for i wheels
C_b_wi = cat(3, C_b_w1, C_b_w2, C_b_w3);
ri = [r1 r2 r3];
I_wi = cat(3, I_w1, I_w2, I_w3);
mi = [mw1 mw2 mw3];

% total inertia
% summation term
I_RWsum = 0;
for i = 1:num_wheels
    I_RWsum = I_RWsum + ...
```

```

        C_b_wi(:, :, i) * ...
        (I_wi(:, :, i) - mi(i)*skew(ri(:, i))*skew(ri(:, i))) ...
        *C_b_wi(:, :, i)';
end
I_tot = I_sc + I_RWsum;

```

control

```

zeta = 0.65;
Ts = 100; % sec

wn = -log(0.02*sqrt(1 - zeta^2))/(zeta*Ts);

Kp = 2*wn^2*I_tot;
Kd = 2*zeta*wn*I_tot;

K = [Kp Kd];

% command to zero attitude
q_C = [0;0;0;1];

```

initial kinematics from part 2

load constants

```

orbitsConstants;

% orbit props
hmag = 53335.2;           % km^2/s
ecc = 0;                  % unitless
raan = 0*pi/180;          % rad
inc = 98.43*pi/180;       % rad
argp = 0*pi/180;          % rad
TA = 0*pi/180;            % rad

% initial body ang velocity wrt ECI, in body components
w0_b_ECI = [0.001; -0.001; 0.002]; % rad/s

% initial wheel spin speeds relative to spacecraft body [rad/s]
% one scalar spin speed per wheel
w0_wi_ECI = zeros(num_wheels,1);

% external torque free
T = [0;0;0];

% initial state vector in ECI
[r0_ECI, v0_ECI] = coesToState(hmag, ecc, TA, raan, inc, argp, muE);

% orbit period
orbParams = getMoreOrbitalParams(hmag, ecc, TA, raan, inc, argp, muE);
orbital_period = orbParams.period;

% initial attitude aligned with LVLH

```

```

phi0    = 0;
theta0 = 0;
psi0    = 0;
Eul0_b_lvlh = [phi0; theta0; psi0];

% initial quaternion relating body to LVLH
q0_b_lvlh = [0;0;0;1];

% DCM relating LVLH to ECI
% rows are LVLH basis vectors written in ECI components
C_LVLH_ECI_0 = LVLH_from_ECI(r0_ECI, v0_ECI);

% DCM relating body to LVLH
C_b_LVLH_0 = build_eulerDCM(3, 2, 1, phi0, theta0, psi0, false);

% DCM relating body to ECI
C_b_ECI_0 = C_b_LVLH_0 * C_LVLH_ECI_0;

% initial Euler angles relating body to ECI using 3-2-1 convention
[phi_b_ECI_0, theta_b_ECI_0, psi_b_ECI_0] = find_321_angles(C_b_ECI_0,
false);
Eul0_b_ECI = [phi_b_ECI_0; theta_b_ECI_0; psi_b_ECI_0];

% initial quaternion relating body to ECI
q0_b_ECI = DCM_to_quat(C_b_ECI_0);

```

Simulate

bus object to pass structure as input

```

Simulink.Bus.createObject(orbParams);

% 300 second case
tstop = 300;
out_300 = sim("SS_Part6.slx");

% five-orbit normal operations case
tstop = 5*orbital_period;
out_5orb = sim("SS_Part6.slx");

```

Plot 5 orbit case

```

% choose which case to plot
out = out_5orb;

% Extract saved outputs
t = out.tout;

E_b_ECI = squeeze(out.E_b_ECI.signals.values);
q_b_ECI = squeeze(out.q_b_ECI.signals.values);
w_b_ECI = squeeze(out.w_b_ECI.signals.values);
M_c      = squeeze(out.M_c.signals.values);
w_wi     = squeeze(out.w_wi_ECI.signals.values);

```

```

% Compute LVLH quantities from existing ECI outputs
E_b_LVLH = zeros(length(t),3);
q_b_LVLH = zeros(length(t),4);
w_b_LVLH = zeros(length(t),3);

n_orb = orbParams.n;

for k = 1:length(t)

    % Circ orbit
    TA_k = TA + n_orb*t(k);

    [r_ECI, v_ECI] = coesToState(hmag, ecc, TA_k, raan, inc, argp, muE);

    C_LVLH_ECI = LVLH_from_ECI(r_ECI, v_ECI);

    C_b_ECI = quat_to_DCM(q_b_ECI(k,:).');

    C_b_LVLH = C_b_ECI * C_LVLH_ECI';

    [phi_L, theta_L, psi_L] = find_321_angles(C_b_LVLH, false);
    E_b_LVLH(k,:) = [phi_L, theta_L, psi_L];

    q_b_LVLH(k,:) = DCM_to_quat(C_b_LVLH).';

    h_ECI = cross(r_ECI, v_ECI);
    w_LVLH_ECI_ECI = h_ECI / norm(r_ECI)^2;

    w_b_LVLH(k,:) = (w_b_ECI(k,:).' - C_b_ECI*w_LVLH_ECI_ECI).';
end

```

wheel speed

```

figure;
plot(t, w_wi);
grid on;
xlabel('Time [s]');
ylabel('Commanded Wheel Speed [rad/s]');
legend('\Omega_{cmd,1}', '\Omega_{cmd,2}', '\Omega_{cmd,3}');
title('Commanded Wheel Speeds');

```

Euler angles: body to ECI

```

figure;
plot(t, rad2deg(E_b_ECI));
grid on;
xlabel('Time [s]');
ylabel('Euler Angles [deg]');
legend('\phi', '\theta', '\psi');
title('Euler Angles: Body to ECI');

```

Quaternion: body to ECI

```
figure;
plot(t, q_b_ECI);
grid on;
xlabel('Time [s]');
ylabel('Quaternion');
legend('q_1', 'q_2', 'q_3', 'q_4');
title('Quaternion: Body to ECI');
```

Euler angles: body to LVLH

```
figure;
plot(t, rad2deg(E_b_LVLH));
grid on;
xlabel('Time [s]');
ylabel('Euler Angles [deg]');
legend('\phi', '\theta', '\psi');
title('Euler Angles: Body to LVLH');
```

Quaternion: body to LVLH

```
figure;
plot(t, q_b_LVLH);
grid on;
xlabel('Time [s]');
ylabel('Quaternion');
legend('q_1', 'q_2', 'q_3', 'q_4');
title('Quaternion: Body to LVLH');
```

Angular velocity wrt ECI, body components

```
figure;
plot(t, w_b_ECI);
grid on;
xlabel('Time [s]');
ylabel('\omega_{b/ECI}^b [rad/s]');
legend('\omega_x', '\omega_y', '\omega_z');
title('Angular Velocity wrt ECI, Body Components');
```

Angular velocity wrt LVLH, body components

```
figure;
plot(t, w_b_LVLH);
grid on;
xlabel('Time [s]');
ylabel('\omega_{b/LVLH}^b [rad/s]');
legend('\omega_x', '\omega_y', '\omega_z');
title('Angular Velocity wrt LVLH, Body Components');
```

Commanded moment

```
figure;
plot(t, M_c);
grid on;
xlabel('Time [s]');
ylabel('M_c [N m]');
legend('M_x', 'M_y', 'M_z');
title('Commanded Moment');
```

Plot 300 second case

```
% choose which case to plot
out = out_300;

% Extract saved outputs
t = out.tout;

E_b_ECI = squeeze(out.E_b_ECI.signals.values);
q_b_ECI = squeeze(out.q_b_ECI.signals.values);
w_b_ECI = squeeze(out.w_b_ECI.signals.values);
M_c      = squeeze(out.M_c.signals.values);
w_wi     = squeeze(out.w_wi_ECI.signals.values);

% Compute LVLH quantities from existing ECI outputs
E_b_LVLH = zeros(length(t), 3);
q_b_LVLH = zeros(length(t), 4);
w_b_LVLH = zeros(length(t), 3);

n_orb = orbParams.n;

for k = 1:length(t)

    % Circ orbit
    TA_k = TA + n_orb*t(k);

    [r_ECI, v_ECI] = coesToState(hmag, ecc, TA_k, raan, inc, argp, muE);

    C_LVLH_ECI = LVLH_from_ECI(r_ECI, v_ECI);

    C_b_ECI = quat_to_DCM(q_b_ECI(k, :).');

    C_b_LVLH = C_b_ECI * C_LVLH_ECI';

    [phi_L, theta_L, psi_L] = find_321_angles(C_b_LVLH, false);
    E_b_LVLH(k, :) = [phi_L, theta_L, psi_L];

    q_b_LVLH(k, :) = DCM_to_quat(C_b_LVLH).';

    h_ECI = cross(r_ECI, v_ECI);
    w_LVLH_ECI_ECI = h_ECI / norm(r_ECI)^2;
```

```
w_b_LVLH(k,:) = (w_b_ECI(k,:).' - C_b_ECI*w_LVLH_ECI_ECI).';  
end
```

wheel speed

```
figure;  
plot(t, w_wi);  
grid on;  
xlabel('Time [s]');  
ylabel('Commanded Wheel Speed [rad/s]');  
legend('\Omega_{cmd,1}', '\Omega_{cmd,2}', '\Omega_{cmd,3}');  
title('Commanded Wheel Speeds');
```

Euler angles: body to ECI

```
figure;  
plot(t, rad2deg(E_b_ECI));  
grid on;  
xlabel('Time [s]');  
ylabel('Euler Angles [deg]');  
legend('\phi', '\theta', '\psi');  
title('Euler Angles: Body to ECI');
```

Quaternion: body to ECI

```
figure;  
plot(t, q_b_ECI);  
grid on;  
xlabel('Time [s]');  
ylabel('Quaternion');  
legend('q_1', 'q_2', 'q_3', 'q_4');  
title('Quaternion: Body to ECI');
```

Euler angles: body to LVLH

```
figure;  
plot(t, rad2deg(E_b_LVLH));  
grid on;  
xlabel('Time [s]');  
ylabel('Euler Angles [deg]');  
legend('\phi', '\theta', '\psi');  
title('Euler Angles: Body to LVLH');
```

Quaternion: body to LVLH

```
figure;  
plot(t, q_b_LVLH);  
grid on;  
xlabel('Time [s]');  
ylabel('Quaternion');
```

```
legend('q_1','q_2','q_3','q_4');  
title('Quaternion: Body to LVLH');
```

Angular velocity wrt ECI, body components

```
figure;  
plot(t, w_b_ECI);  
grid on;  
xlabel('Time [s]');  
ylabel('\omega_{b/ECI}^b [rad/s]');  
legend('\omega_x', '\omega_y', '\omega_z');  
title('Angular Velocity wrt ECI, Body Components');
```

Angular velocity wrt LVLH, body components

```
figure;  
plot(t, w_b_LVLH);  
grid on;  
xlabel('Time [s]');  
ylabel('\omega_{b/LVLH}^b [rad/s]');  
legend('\omega_x', '\omega_y', '\omega_z');  
title('Angular Velocity wrt LVLH, Body Components');
```

Commanded moment

```
figure;  
plot(t, M_c);  
grid on;  
xlabel('Time [s]');  
ylabel('M_c [N m]');  
legend('M_x', 'M_y', 'M_z');  
title('Commanded Moment');
```

Published with MATLAB® R2024b